```
import re
import numpy as np
import pandas as pd

FILE_PATH = r"Green Software Engineering Maturity Level Assessment (responses) (1) (1).xlsx"
SHEET_NAME = "Form responses 1"  # in your file, this is the one

ITEM_COL_RE = re.compile(r"^\s*\d+\s*-")

# Dimensions as sequential blocks (5 items each) — exactly what you ran in the print
DIMENSIONS_MAP = {
    "D1": [1, 2, 3, 4, 5],
    "D2": [6, 7, 8, 9, 10],
    "D3": [11, 12, 13, 14, 15],
    "D4": [16, 17, 18, 19, 20],
    "D5": [21, 22, 23, 24, 25],
    "D6": [26, 27, 28, 29, 30],
    "D7": [31, 32, 33, 34, 35],
}

def parse_cell(x):
    """Converts '3 - Partially' -> 3. If it fails -> NaN."""
    if pd.isna(x):
        return np.nan
    if isinstance(x, (int, float, np.integer, np.floating)):
        return float(x)
    s = str(x).strip()
    m = re.match(r"^\s*(\d+)", s)
    return float(m.group(1)) if m else np.nan

def extract_items(df: pd.DataFrame) -> pd.DataFrame:
    item_cols = [c for c in df.columns if ITEM_COL_RE.match(str(c))]
    if not item_cols:
        raise ValueError("No item columns found matching the pattern '1 - ...'.")

    items_num = df[item_cols].apply(lambda col: col.map(parse_cell))
    return items_num.dropna(axis=0, how="any")  # complete cases

def build_dimensions(items_num: pd.DataFrame, dim_map: dict) -> pd.DataFrame:
    """Creates 7 columns (D1..D7): each one is the mean of the items defined in the map."""
    col_by_number = {}
    for c in items_num.columns:
        m = re.match(r"^\s*(\d+)\s*-", str(c))
        if m:
            col_by_number[int(m.group(1))] = c

    dims = {}
```

```
    for dim_name, nums in dim_map.items():
        missing = [n for n in nums if n not in col_by_number]
        if missing:
            raise ValueError(f"{dim_name}: items not found: {missing}")
        cols = [col_by_number[n] for n in nums]
        dims[dim_name] = items_num[cols].mean(axis=1)

    return pd.DataFrame(dims)

def cronbach_alpha_classic(df_dim: pd.DataFrame) -> dict:
    """
    α = k/(k−1) * (1 − (Σ s_i² / s_T²))
    s_T² computed over the total score = sum of the 7 dimensions.
    """
    df_dim = df_dim.dropna(axis=0, how="any")
    N, k = df_dim.shape
    if N < 2 or k < 2:
        raise ValueError("Requires N>=2 and k>=2.")

    variances = df_dim.var(axis=0, ddof=1)
    sum_s2 = float(variances.sum())

    total_score = df_dim.sum(axis=1)
    s2_T = float(total_score.var(ddof=1))
    if s2_T == 0:
        raise ValueError("Total variance is zero; alpha undefined.")

    alpha = (k / (k - 1)) * (1 - (sum_s2 / s2_T))
    ratio = sum_s2 / s2_T

    # r̄ computed in two ways:
    # (1) "actual": mean of inter-dimension correlations
    corr = df_dim.corr()
    rbar_corr = float(corr.values[np.triu_indices(k, 1)].mean())

    # (2) using the expression from your text (with α rounded to 2 decimals)
    alpha_2 = round(alpha, 2)
    rbar_expr = float(alpha_2 / (k - alpha_2 * (k - 1)))

    return {
        "N": int(N),
        "k": int(k),
        "sum_s2": sum_s2,
        "s2_T": s2_T,
        "alpha": float(alpha),
        "alpha_2": float(alpha_2),
        "ratio": float(ratio),
        "rbar_corr": float(rbar_corr),
```

```
        "rbar_expr": float(rbar_expr),
    }

def main():
    df = pd.read_excel(FILE_PATH, sheet_name=SHEET_NAME)
    items = extract_items(df)
    dim_df = build_dimensions(items, DIMENSIONS_MAP)
    res = cronbach_alpha_classic(dim_df)

    print("\n=== MEANS PER DIMENSION (7 means) — CLASSIC CALCULATION ===")
    print(f"N (valid cases): {res['N']}")
    print(f"k: {res['k']}")
    print(f"Σ s²_i (sum of variances of the 7 dimensions): {res['sum_s2']:.6f}")
    print(f"s²_T (variance of the total score): {res['s2_T']:.6f}")
    print(f"α (Cronbach, raw): {res['alpha']:.6f}")
    print(f"α (Cronbach, 2 decimals): {res['alpha_2']:.2f}")
    print(f"Σ s²_i / s²_T: {res['ratio']:.6f}")
    print(f"r̄ (mean via correlation): {res['rbar_corr']:.6f}")
    print(f"r̄ (via expression using rounded α): {res['rbar_expr']:.2f}")

if __name__ == "__main__":
    main()
```